Lab Sheet 11

Real-Time Mobile Camera Integration via Python & OpenCV
Computer Vision Lab

Objective

To stream live video from a mobile phone camera to a PC, process the incoming feed using Python and OpenCV, and apply real-time computer vision techniques (motion detection, QR scanning, gesture control, face recognition, document scanning, colour inspection, air painting, OCR, drowsiness detection and AR overlays).

Initial Setup (common to all tasks)

1. Connect the phone camera

- Install **IP Webcam** (Android) or **iVCam / EpocCam** (iOS/Android) on the phone.
- Connect the phone and the PC to the **same Wi-Fi network**.
- In IP Webcam press *Start server*. It shows an address such as `http://192.168.1.5:8080`. The video stream is at `/video`.
- Check the stream in a PC browser first. For iVCam/EpocCam the phone appears as a normal webcam, so use `cv2.VideoCapture(0)` (or 1, 2) instead of the URL.

2. Install libraries

```
pip install opencv-python numpy mediapipe pyautogui pyzbar face_recognition pytesseract pygame
# Tesseract OCR engine must also be installed separately (Task 8)
# face_recognition needs dlib (CMake + C++ build tools)
```

3. Common code to open the stream

```
import cv2
SOURCE = "http://192.168.1.5:8080/video"   # change to your phone's IP (or use 0 for iVCam)
cap = cv2.VideoCapture(SOURCE)
if not cap.isOpened():
    raise SystemExit("Cannot open camera stream - check IP / Wi-Fi")
while True:
    ret, frame = cap.read()
    if not ret: break
    cv2.imshow("Mobile Feed", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'): break
cap.release(); cv2.destroyAllWindows()
```

Every program below follows this same capture loop. Press **q** to quit unless stated otherwise. The programs were written to be run on a PC with the phone streaming; tune thresholds (marked in the code) to your lighting and camera.

Task 1: Live Motion & Security Alarm Detector

Aim: Detect motion in the live stream, show an on-screen alert and save a snapshot to disk.

Key skills: cv2.absdiff(), frame subtraction, contour area thresholding.

Algorithm

- Convert each frame to grayscale and blur it to remove noise.
- Compute `cv2.absdiff` between the previous and current frame, threshold and dilate it.
- Find contours; ignore those with area below MIN_AREA.
- If any contour is large enough: draw boxes, display "MOTION DETECTED" and save a time-stamped snapshot (at most once every 3 s).

Program

```
import cv2, os, time
from datetime import datetime
```

```
SOURCE = "http://192.168.1.5:8080/video"
MIN_AREA = 1500            # pixel-area threshold for "real" motion
os.makedirs("snapshots", exist_ok=True)

cap = cv2.VideoCapture(SOURCE)
ret, first = cap.read()
prev = cv2.GaussianBlur(cv2.cvtColor(first, cv2.COLOR_BGR2GRAY), (21, 21), 0)
last_save = 0

while True:
    ret, frame = cap.read()
    if not ret: break
    gray = cv2.GaussianBlur(cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY), (21, 21), 0)

    diff = cv2.absdiff(prev, gray)
    thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)[1]
    thresh = cv2.dilate(thresh, None, iterations=2)
    cnts, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    motion = False
    for c in cnts:
        if cv2.contourArea(c) < MIN_AREA:
            continue
        motion = True
        x, y, w, h = cv2.boundingRect(c)
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

    if motion:
        cv2.putText(frame, "MOTION DETECTED!", (15, 40),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 3)
        if time.time() - last_save > 3:
            name = datetime.now().strftime("snapshots/alert_%Y%m%d_%H%M%S.jpg")
            cv2.imwrite(name, frame)
            last_save = time.time()

    prev = gray
    cv2.imshow("Security Alarm", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'): break

cap.release(); cv2.destroyAllWindows()
```

Output / Notes: Moving objects get green boxes and a red alert; snapshots appear in the *snapshots* folder. Increase MIN_AREA if small noise triggers alarms.

Task 2: Mobile QR Code & Barcode Scanner for Inventory

Aim: Decode QR codes and barcodes from the stream and log each item to a CSV file with a timestamp.

Key skills: OpenCV QRCodeDetector / pyzbar, CSV file handling.

Algorithm

- Use `cv2.QRCodeDetector().detectAndDecode()` for QR codes; use pyzbar for barcodes (EAN, Code128, etc.) and QR.
- Draw the outline and decoded text on the frame.
- Append (timestamp, type, data) to *inventory_log.csv*; ignore the same code if it was logged in the last 5 s to avoid duplicates.

Program

```
import cv2, csv, os, time
from datetime import datetime
try:
    from pyzbar import pyzbar
except ImportError:
    pyzbar = None                      # QR-only mode if pyzbar is missing

SOURCE = "http://192.168.1.5:8080/video"
LOG = "inventory_log.csv"
if not os.path.exists(LOG):
    with open(LOG, "w", newline="") as f:
        csv.writer(f).writerow(["Timestamp", "Type", "Data"])

def log_item(kind, data):
    with open(LOG, "a", newline="") as f:
        csv.writer(f).writerow([datetime.now().strftime("%Y-%m-%d %H:%M:%S"), kind, data])
```

```
qr = cv2.QRCodeDetector()
cap = cv2.VideoCapture(SOURCE)
last_seen = {}                          # data -> time last logged

while True:
    ret, frame = cap.read()
    if not ret: break
    found = []

    data, pts, _ = qr.detectAndDecode(frame)
    if data:
        found.append(("QRCODE", data))
        if pts is not None:
            pts = pts.astype(int).reshape(-1, 2)
            cv2.polylines(frame, [pts], True, (0, 255, 0), 3)

    if pyzbar:
        for obj in pyzbar.decode(frame):
            text = obj.data.decode("utf-8")
            if obj.type != "QRCODE" or text != data:
                found.append((obj.type, text))
            x, y, w, h = obj.rect
            cv2.rectangle(frame, (x, y), (x + w, y + h), (255, 0, 0), 2)

    for kind, text in found:
        cv2.putText(frame, f"{kind}: {text}", (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
                    0.7, (0, 0, 255), 2)
        if time.time() - last_seen.get(text, 0) > 5:      # de-duplicate
            log_item(kind, text)
            last_seen[text] = time.time()
            print("Logged:", kind, text)

    cv2.imshow("Inventory Scanner", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'): break

cap.release(); cv2.destroyAllWindows()
```

Output / Notes: Each new scan adds a row such as *2026-10-15 10:32:08, QRCODE, ITEM-0042* to inventory_log.csv.

Task 3: Hand Gesture Desktop Controller

Aim: Detect hand landmarks with MediaPipe, count raised fingers and map them to PC actions.

Key skills: MediaPipe Hands, gesture mapping, pyautogui.

Algorithm

- Run MediaPipe Hands on each RGB frame to get 21 landmarks.
- A finger is "up" if its tip (landmarks 8, 12, 16, 20) is above its PIP joint (6, 10, 14, 18); the thumb (tip 4) is compared on the x-axis using the left/right label.
- Map the finger count to actions: 1 = scroll up, 2 = scroll down, 3 = next slide, 4 = previous slide, 5 = volume up, 0 (fist) = mute.
- Use a 0.8 s cooldown so one gesture does not fire repeatedly.

Program

```
import cv2, time
import mediapipe as mp
import pyautogui

SOURCE = "http://192.168.1.5:8080/video"
mp_hands, mp_draw = mp.solutions.hands, mp.solutions.drawing_utils
hands = mp_hands.Hands(max_num_hands=1, min_detection_confidence=0.7,
                       min_tracking_confidence=0.6)
TIPS, PIPS = [8, 12, 16, 20], [6, 10, 14, 18]

def count_fingers(lm, label):
    up = 0
    # thumb: compare x of tip (4) and IP joint (3), direction depends on hand
    if label == "Right":
        up += lm[4].x < lm[3].x
    else:
        up += lm[4].x > lm[3].x
    for t, p in zip(TIPS, PIPS):
```

```
        up += lm[t].y < lm[p].y
    return int(up)

def act(n):
    if n == 1:   pyautogui.scroll(300)
    elif n == 2: pyautogui.scroll(-300)
    elif n == 3: pyautogui.press("right")          # next slide
    elif n == 4: pyautogui.press("left")           # previous slide
    elif n == 5: pyautogui.press("volumeup")
    elif n == 0: pyautogui.press("volumemute")

cap = cv2.VideoCapture(SOURCE)
last = 0
while True:
    ret, frame = cap.read()
    if not ret: break
    frame = cv2.flip(frame, 1)                     # mirror view
    res = hands.process(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))

    if res.multi_hand_landmarks:
        hand = res.multi_hand_landmarks[0]
        label = res.multi_handedness[0].classification[0].label
        mp_draw.draw_landmarks(frame, hand, mp_hands.HAND_CONNECTIONS)
        n = count_fingers(hand.landmark, label)
        cv2.putText(frame, f"Fingers: {n}", (10, 40),
                    cv2.FONT_HERSHEY_SIMPLEX, 1.2, (0, 255, 0), 3)
        if time.time() - last > 0.8:
            act(n); last = time.time()

    cv2.imshow("Gesture Controller", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'): break

cap.release(); cv2.destroyAllWindows()
```

Output / Notes: Open a PDF/PowerPoint or web page before testing. pyautogui.FAILSAFE is on: moving the mouse to a screen corner aborts the program. The thumb rule may need to be swapped if the mirrored view reverses left/right.

Task 4: Smart Attendance System via Face Recognition

Aim: Detect faces in the stream, match them with known faces in a local folder and mark attendance automatically.

Key skills: face_recognition library, facial feature embeddings (128-d), face distance matching.

Algorithm

- Put one clear photo per person in *known_faces/* named *Name.jpg*; compute and store their 128-d encodings.
- For each frame (downscaled to 1/4 for speed) detect faces and compute encodings.
- Compare with known encodings using `face_distance`; the best match with distance < 0.5 is accepted.
- Write name, date and time to *attendance.csv* once per person per day-session.

Program

```
import cv2, os, csv
import numpy as np
import face_recognition
from datetime import datetime

SOURCE = "http://192.168.1.5:8080/video"
KNOWN_DIR, FILE = "known_faces", "attendance.csv"

known_enc, known_names = [], []
for f in os.listdir(KNOWN_DIR):
    img = face_recognition.load_image_file(os.path.join(KNOWN_DIR, f))
    enc = face_recognition.face_encodings(img)
    if enc:
        known_enc.append(enc[0])
        known_names.append(os.path.splitext(f)[0])
print("Loaded:", known_names)

if not os.path.exists(FILE):
    with open(FILE, "w", newline="") as fh:
        csv.writer(fh).writerow(["Name", "Date", "Time"])
```

```
marked = set()

def mark(name):
    if name in marked: return
    now = datetime.now()
    with open(FILE, "a", newline="") as fh:
        csv.writer(fh).writerow([name, now.strftime("%Y-%m-%d"), now.strftime("%H:%M:%S")])
    marked.add(name)
    print("Attendance marked:", name)

cap = cv2.VideoCapture(SOURCE)
while True:
    ret, frame = cap.read()
    if not ret: break
    small = cv2.resize(frame, (0, 0), fx=0.25, fy=0.25)
    rgb = cv2.cvtColor(small, cv2.COLOR_BGR2RGB)
    locs = face_recognition.face_locations(rgb)
    encs = face_recognition.face_encodings(rgb, locs)

    for enc, (top, right, bottom, left) in zip(encs, locs):
        name = "Unknown"
        if known_enc:
            d = face_recognition.face_distance(known_enc, enc)
            best = int(np.argmin(d))
            if d[best] < 0.5:                    # tolerance
                name = known_names[best]
                mark(name)
        top, right, bottom, left = top * 4, right * 4, bottom * 4, left * 4
        color = (0, 255, 0) if name != "Unknown" else (0, 0, 255)
        cv2.rectangle(frame, (left, top), (right, bottom), color, 2)
        cv2.putText(frame, name, (left, top - 8), cv2.FONT_HERSHEY_SIMPLEX, 0.8, color, 2)

    cv2.imshow("Smart Attendance", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'): break

cap.release(); cv2.destroyAllWindows()
```

Output / Notes: Known faces get a green box with their name; unknown faces a red box. Use well-lit front-facing reference photos and get consent from the people enrolled.

Task 5: Document Scanner with Perspective Transform

Aim: Detect the four corners of a sheet on a table, flatten it with a perspective warp and convert it to a high-contrast black-and-white scan.

Key skills: Canny edge detection, cv2.findContours(), cv2.getPerspectiveTransform().

Algorithm

- Grayscale, blur and run Canny edge detection; dilate to close gaps.
- Find the largest contours and approximate each with `approxPolyDP`; the first with 4 points is the document.
- Order the corners (top-left, top-right, bottom-right, bottom-left).
- Compute the perspective matrix and warp to a flat rectangle; apply adaptive thresholding for the B/W scan.
- Press **s** to save the scan, **q** to quit.

Program

```
import cv2, numpy as np
from datetime import datetime

SOURCE = "http://192.168.1.5:8080/video"

def order_points(pts):
    rect = np.zeros((4, 2), dtype="float32")
    s = pts.sum(axis=1); d = np.diff(pts, axis=1)
    rect[0] = pts[np.argmin(s)]     # top-left
    rect[2] = pts[np.argmax(s)]     # bottom-right
    rect[1] = pts[np.argmin(d)]     # top-right
    rect[3] = pts[np.argmax(d)]     # bottom-left
    return rect

def four_point_warp(img, pts):
    tl, tr, br, bl = order_points(pts)
    w = int(max(np.linalg.norm(br - bl), np.linalg.norm(tr - tl)))
```

```
    h = int(max(np.linalg.norm(tr - br), np.linalg.norm(tl - bl)))
    dst = np.array([[0, 0], [w - 1, 0], [w - 1, h - 1], [0, h - 1]], dtype="float32")
    M = cv2.getPerspectiveTransform(np.array([tl, tr, br, bl], dtype="float32"), dst)
    return cv2.warpPerspective(img, M, (w, h))

def find_document(frame):
    gray = cv2.GaussianBlur(cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY), (5, 5), 0)
    edges = cv2.dilate(cv2.Canny(gray, 75, 200), None, iterations=1)
    cnts, _ = cv2.findContours(edges, cv2.RETR_LIST, cv2.CHAIN_APPROX_SIMPLE)
    for c in sorted(cnts, key=cv2.contourArea, reverse=True)[:5]:
        approx = cv2.approxPolyDP(c, 0.02 * cv2.arcLength(c, True), True)
        if len(approx) == 4 and cv2.contourArea(approx) > 20000:
            return approx.reshape(4, 2)
    return None

cap = cv2.VideoCapture(SOURCE)
while True:
    ret, frame = cap.read()
    if not ret: break
    view = frame.copy()
    doc = find_document(frame)
    if doc is not None:
        cv2.polylines(view, [doc.astype(int)], True, (0, 255, 0), 3)
        cv2.putText(view, "Document found - press 's' to scan", (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
    cv2.imshow("Document Scanner", view)

    key = cv2.waitKey(1) & 0xFF
    if key == ord('s') and doc is not None:
        warped = four_point_warp(frame, doc)
        gray = cv2.cvtColor(warped, cv2.COLOR_BGR2GRAY)
        scan = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                     cv2.THRESH_BINARY, 21, 10)
        name = datetime.now().strftime("scan_%Y%m%d_%H%M%S.png")
        cv2.imwrite(name, scan)
        cv2.imshow("Scan", scan)
        print("Saved", name)
    elif key == ord('q'):
        break

cap.release(); cv2.destroyAllWindows()
```

Output / Notes: Place the sheet on a darker, plain background so the edges are clear.

Task 6: Portable Color & Defect Inspector

Aim: Isolate colour ranges in HSV to classify objects (e.g. ripe red vs unripe green fruit) and flag defects.

Key skills: RGB to HSV conversion, cv2.inRange, masking, morphology.

Algorithm

- Convert BGR frame to HSV (hue is less sensitive to lighting than RGB).
- Create a mask for each target colour using `cv2.inRange` (red needs two hue ranges because it wraps around 0/180).
- Clean masks with morphological open/close; find contours above a minimum area.
- Label each object (RIPE / UNRIPE) and show counts; objects that are neither colour but large can be flagged as DEFECT.

Program

```
import cv2, numpy as np

SOURCE = "http://192.168.1.5:8080/video"
MIN_AREA = 800

# HSV ranges (tune with your lighting)
RED1  = (np.array([0, 120, 70]),   np.array([10, 255, 255]))
RED2  = (np.array([170, 120, 70]), np.array([180, 255, 255]))
GREEN = (np.array([35, 80, 60]),   np.array([85, 255, 255]))
kernel = np.ones((5, 5), np.uint8)

def clean(mask):
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
    return cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
```

```
def mark(frame, mask, label, color):
    cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    n = 0
    for c in cnts:
        if cv2.contourArea(c) < MIN_AREA: continue
        n += 1
        x, y, w, h = cv2.boundingRect(c)
        cv2.rectangle(frame, (x, y), (x + w, y + h), color, 2)
        cv2.putText(frame, label, (x, y - 6), cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)
    return n

cap = cv2.VideoCapture(SOURCE)
while True:
    ret, frame = cap.read()
    if not ret: break
    hsv = cv2.cvtColor(cv2.GaussianBlur(frame, (7, 7), 0), cv2.COLOR_BGR2HSV)

    red = clean(cv2.inRange(hsv, *RED1) | cv2.inRange(hsv, *RED2))
    green = clean(cv2.inRange(hsv, *GREEN))

    n_ripe = mark(frame, red, "RIPE", (0, 0, 255))
    n_unripe = mark(frame, green, "UNRIPE", (0, 200, 0))

    cv2.putText(frame, f"Ripe: {n_ripe}  Unripe: {n_unripe}", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)
    cv2.imshow("Inspector", frame)
    cv2.imshow("Red mask", red)
    if cv2.waitKey(1) & 0xFF == ord('q'): break

cap.release(); cv2.destroyAllWindows()
```

Output / Notes: For defect detection on parts, set the HSV range of the good colour and report any contour/hole that falls outside it. Use trackbars (cv2.createTrackbar) to tune ranges live.

Task 7: Virtual Whiteboard / Air Painting App

Aim: Track a coloured object (blue marker cap) and draw its path on a canvas to write in mid-air.

Key skills: Colour-based object tracking, cv2.line / cv2.circle, canvas buffer.

Algorithm

- Flip the frame (mirror view) and convert to HSV; threshold the blue range.
- Take the largest contour and its minimum enclosing circle as the pen tip.
- Join the previous and current tip positions with `cv2.line` on a separate black canvas.
- Overlay the canvas on the video; **c** clears, **s** saves, **q** quits.

Program

```
import cv2, numpy as np
from datetime import datetime

SOURCE = "http://192.168.1.5:8080/video"
LOWER, UPPER = np.array([100, 150, 50]), np.array([130, 255, 255])   # blue
kernel = np.ones((5, 5), np.uint8)

cap = cv2.VideoCapture(SOURCE)
canvas, prev = None, None

while True:
    ret, frame = cap.read()
    if not ret: break
    frame = cv2.flip(frame, 1)
    if canvas is None:
        canvas = np.zeros_like(frame)

    hsv = cv2.cvtColor(cv2.GaussianBlur(frame, (11, 11), 0), cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv, LOWER, UPPER)
    mask = cv2.morphologyEx(cv2.erode(mask, kernel, iterations=1), cv2.MORPH_OPEN, kernel)
    cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    pt = None
    if cnts:
        c = max(cnts, key=cv2.contourArea)
        (x, y), r = cv2.minEnclosingCircle(c)
        if r > 10:
```

```
            pt = (int(x), int(y))
            cv2.circle(frame, pt, int(r), (0, 255, 255), 2)

    if pt is not None and prev is not None:
        cv2.line(canvas, prev, pt, (0, 0, 255), 6)       # red ink
    prev = pt                                            # lift pen if object lost

    painted = canvas.any(axis=2)
    frame[painted] = canvas[painted]                     # overlay ink
    cv2.imshow("Air Painting", frame)
    cv2.imshow("Canvas", canvas)

    key = cv2.waitKey(1) & 0xFF
    if key == ord('c'): canvas[:] = 0
    elif key == ord('s'): cv2.imwrite(datetime.now().strftime("paint_%H%M%S.png"), canvas)
    elif key == ord('q'): break

cap.release(); cv2.destroyAllWindows()
```

Output / Notes: Use a bright blue object and keep other blue items out of view. Change LOWER/UPPER for another colour.

Task 8: Real-Time OCR Text Reader

Aim: Extract printed text from the live feed with Tesseract and show it as an overlay on the video.

Key skills: pytesseract, image thresholding, cv2.putText.

Algorithm

- Install the Tesseract engine and set `pytesseract.pytesseract.tesseract_cmd` on Windows.
- Convert to grayscale and apply Otsu thresholding to improve contrast.
- OCR is slow, so run it only every 15th frame and reuse the last result in between.
- Use `image_to_data` to get each word's box and confidence; draw confident words and show the full recognized string in a bar at the bottom.

Program

```
import cv2
import pytesseract
from pytesseract import Output

# Windows only - path to tesseract.exe:
# pytesseract.pytesseract.tesseract_cmd = r"C:\Program Files\Tesseract-OCR\tesseract.exe"

SOURCE = "http://192.168.1.5:8080/video"
OCR_EVERY = 15
CONF_MIN = 60

cap = cv2.VideoCapture(SOURCE)
n, words, line = 0, [], ""

while True:
    ret, frame = cap.read()
    if not ret: break
    n += 1

    if n % OCR_EVERY == 0:
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        gray = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)[1]
        d = pytesseract.image_to_data(gray, config="--oem 3 --psm 6",
                                      output_type=Output.DICT)
        words = []
        for i, txt in enumerate(d["text"]):
            if txt.strip() and float(d["conf"][i]) > CONF_MIN:
                words.append((txt, d["left"][i], d["top"][i], d["width"][i], d["height"][i]))
        line = " ".join(w[0] for w in words)

    for txt, x, y, w, h in words:
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
        cv2.putText(frame, txt, (x, y - 5), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)

    # text bar at bottom
    cv2.rectangle(frame, (0, frame.shape[0] - 40), (frame.shape[1], frame.shape[0]), (0, 0, 0), -1)
    cv2.putText(frame, line[:80], (10, frame.shape[0] - 12),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)
```

```
    cv2.imshow("OCR Reader", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'): break

cap.release(); cv2.destroyAllWindows()
```

Output / Notes: Works best with large, sharp, evenly-lit printed text held parallel to the camera. Use the phone's focus/torch settings if the text is blurry.

Task 9: Mobile Driver Drowsiness / Blink Monitor

Aim: Compute the Eye Aspect Ratio (EAR) from face landmarks and sound an alarm if the eyes stay closed for too long.

Key skills: MediaPipe Face Mesh, Euclidean distance, audio trigger (winsound / pygame).

Algorithm

- Get 468 face-mesh landmarks and pick the six points of each eye (p1..p6).
- EAR = (|p2 - p6| + |p3 - p5|) / (2 |p1 - p4|). It is about 0.3 for open eyes and falls toward 0 when the eye closes.
- Average the two eyes. If EAR < 0.21 continuously for more than 2 seconds, show DROWSINESS ALERT and play a beep.
- Reset the timer as soon as the eyes reopen.

Program

```
import cv2, time, math
import mediapipe as mp

try:
    import winsound
    def beep(): winsound.Beep(1800, 400)
except ImportError:                                  # Linux / macOS fallback
    def beep(): print("\a", end="", flush=True)

SOURCE = "http://192.168.1.5:8080/video"
EAR_THRESH, CLOSED_SECONDS = 0.21, 2.0
# MediaPipe indices: p1, p2, p3, p4, p5, p6
LEFT  = [362, 385, 387, 263, 373, 380]
RIGHT = [33, 160, 158, 133, 153, 144]

face_mesh = mp.solutions.face_mesh.FaceMesh(max_num_faces=1, refine_landmarks=True)

def dist(a, b): return math.hypot(a[0] - b[0], a[1] - b[1])

def ear(pts):
    p1, p2, p3, p4, p5, p6 = pts
    return (dist(p2, p6) + dist(p3, p5)) / (2.0 * dist(p1, p4))

cap = cv2.VideoCapture(SOURCE)
closed_since = None

while True:
    ret, frame = cap.read()
    if not ret: break
    h, w = frame.shape[:2]
    res = face_mesh.process(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))

    if res.multi_face_landmarks:
        lm = res.multi_face_landmarks[0].landmark
        get = lambda idx: [(lm[i].x * w, lm[i].y * h) for i in idx]
        left, right = get(LEFT), get(RIGHT)
        e = (ear(left) + ear(right)) / 2.0
        for x, y in left + right:
            cv2.circle(frame, (int(x), int(y)), 2, (0, 255, 0), -1)
        cv2.putText(frame, f"EAR: {e:.2f}", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 0), 2)

        if e < EAR_THRESH:
            if closed_since is None: closed_since = time.time()
            if time.time() - closed_since > CLOSED_SECONDS:
                cv2.putText(frame, "DROWSINESS ALERT!", (10, 80),
                            cv2.FONT_HERSHEY_SIMPLEX, 1.2, (0, 0, 255), 3)
                beep()
        else:
            closed_since = None
```

```
    else:
        closed_since = None

    cv2.imshow("Drowsiness Monitor", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'): break

cap.release(); cv2.destroyAllWindows()
```

Output / Notes: Calibrate EAR_THRESH for each person by printing EAR with eyes open and closed. Eyeglasses and poor lighting reduce accuracy.

Task 10: Real-Time Augmented Reality (AR) Filter Overlay

Aim: Detect facial landmarks and overlay virtual glasses that follow, scale and rotate with the face using alpha blending.

Key skills: Landmark localization, alpha-channel image overlay.

Algorithm

- Use MediaPipe Face Mesh; take the outer eye corners (landmarks 33 and 263) as anchors.
- Scale the glasses image to about 1.8 times the distance between the anchors and rotate it by the head-tilt angle.
- Centre it at the midpoint of the anchors and blend: output = alpha x glasses + (1 - alpha) x background (alpha from the PNG's 4th channel).
- Clip the overlay at the frame borders so it never crashes when the face is near the edge.

Program

```
import cv2, math
import numpy as np
import mediapipe as mp

SOURCE = "http://192.168.1.5:8080/video"

def make_glasses(w=400, h=140):
    """Fallback BGRA glasses drawn with OpenCV if glasses.png is missing."""
    img = np.zeros((h, w, 4), np.uint8)
    for cx in (100, 300):
        cv2.circle(img, (cx, h // 2), 60, (20, 20, 20, 255), 10)
        cv2.circle(img, (cx, h // 2), 52, (200, 160, 60, 110), -1)
    cv2.line(img, (160, h // 2), (240, h // 2), (20, 20, 20, 255), 10)
    return img

glasses = cv2.imread("glasses.png", cv2.IMREAD_UNCHANGED)      # must have alpha channel
if glasses is None or glasses.shape[2] != 4:
    glasses = make_glasses()

def overlay(bg, fg, cx, cy):
    """Alpha-blend BGRA fg onto BGR bg, centred at (cx, cy), with edge clipping."""
    h, w = fg.shape[:2]
    x, y = cx - w // 2, cy - h // 2
    x1, y1 = max(x, 0), max(y, 0)
    x2, y2 = min(x + w, bg.shape[1]), min(y + h, bg.shape[0])
    if x1 >= x2 or y1 >= y2: return
    part = fg[y1 - y:y2 - y, x1 - x:x2 - x]
    alpha = part[:, :, 3:4] / 255.0
    roi = bg[y1:y2, x1:x2]
    bg[y1:y2, x1:x2] = (alpha * part[:, :, :3] + (1 - alpha) * roi).astype(np.uint8)

face_mesh = mp.solutions.face_mesh.FaceMesh(max_num_faces=1, refine_landmarks=True)
cap = cv2.VideoCapture(SOURCE)

while True:
    ret, frame = cap.read()
    if not ret: break
    h, w = frame.shape[:2]
    res = face_mesh.process(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))

    if res.multi_face_landmarks:
        lm = res.multi_face_landmarks[0].landmark
        p1 = (lm[33].x * w,  lm[33].y * h)        # outer corner, subject's right eye
        p2 = (lm[263].x * w, lm[263].y * h)       # outer corner, subject's left eye
        eye_dist = math.hypot(p2[0] - p1[0], p2[1] - p1[1])
        angle = math.degrees(math.atan2(p2[1] - p1[1], p2[0] - p1[0]))
```

```
            new_w = max(int(eye_dist * 1.8), 10)
            new_h = max(int(glasses.shape[0] * new_w / glasses.shape[1]), 10)
            g = cv2.resize(glasses, (new_w, new_h), interpolation=cv2.INTER_AREA)
            M = cv2.getRotationMatrix2D((new_w / 2, new_h / 2), -angle, 1.0)
            g = cv2.warpAffine(g, M, (new_w, new_h), borderMode=cv2.BORDER_CONSTANT,
                               borderValue=(0, 0, 0, 0))

            cx, cy = int((p1[0] + p2[0]) / 2), int((p1[1] + p2[1]) / 2)
            overlay(frame, g, cx, cy)

        cv2.imshow("AR Filter", frame)
        if cv2.waitKey(1) & 0xFF == ord('q'): break

    cap.release(); cv2.destroyAllWindows()
```

Output / Notes: Use a transparent PNG (RGBA) of glasses/hat as *glasses.png*. The same overlay() function can draw hats (anchor: forehead landmark 10) or a 3D-style bounding box around the face.

Troubleshooting

Problem	Fix
Stream will not open	Same Wi-Fi for phone and PC; check URL in a browser; allow Python through the firewall; try `/video` or `/videofeed`.
High lag in the feed	Lower resolution in the app, process every 2nd frame, or set `cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)`.
Colours not detected	Tune the HSV ranges under your lighting; use trackbars.
mediapipe has no 'solutions'	Use a mediapipe release that still includes the legacy *solutions* API, or port the code to the newer Tasks API.

Conclusion

A mobile phone camera was used as a wireless video source for a PC, and ten real-time computer vision applications were implemented with Python and OpenCV. They demonstrate frame differencing, decoding, landmark-based gesture and face analysis, geometric transforms, colour segmentation, OCR and image blending, showing how low-cost hardware can be used for practical vision systems.